

**LAPORAN ANALISIS TUGAS AKHIR AI
IMPLEMENTASI ALGORITMA DIJKSTRA PADA SISTEM NAVIGASI
RUTE TERPENDEK BERBASIS WEB MAP INTERAKTIF DI
UNIVERSITAS BENGKULU (UNIB SMART ROUTE)**



Disusun Oleh :

Nama : Tri Haiji Januarli

NPM : G1A024069

Kelas : A Informatika

Mata Kuliah :

Artificial Intelligence

Dosen Pengampu :

Ir. Arie Vatesia, S.T., M.T., Ph.D., IPP

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS BENGKULU

2026

1. Pendahuluan

Pada bagian ini dilakukan analisis terhadap sistem navigasi rute terpendek di lingkungan Universitas Bengkulu yang dibangun menggunakan algoritma Dijkstra berbasis web. Analisis ini bertujuan untuk menjelaskan bagaimana setiap komponen dalam sistem bekerja, mulai dari representasi data lokasi, pembentukan graf, proses perhitungan jalur terpendek, hingga hasil visualisasi pada peta digital. Sistem yang dikembangkan tidak hanya berfokus pada perhitungan rute secara teoritis, tetapi juga diimplementasikan dalam bentuk aplikasi interaktif berbasis web menggunakan Flask sebagai backend dan Leaflet.js sebagai media pemetaan.

Oleh karena itu, analisis dilakukan untuk menggambarkan keterkaitan antara struktur data, algoritma yang digunakan, serta hasil implementasi pada antarmuka pengguna. Dengan adanya analisis ini, diharapkan dapat memberikan pemahaman yang lebih jelas mengenai cara kerja sistem dalam menentukan rute optimal serta bagaimana sistem mampu menyajikan informasi navigasi secara efektif kepada pengguna.

2. ANALISIS SOURCE CODE

2.1 Import Library

Source Code :

```
import heapq  
from flask import Flask, jsonify, render_template, request
```

Penjelasan :

Library heapq digunakan untuk mengimplementasikan priority queue pada algoritma Dijkstra. Priority queue berfungsi untuk memilih node dengan jarak terkecil yang akan diproses terlebih dahulu sehingga proses pencarian jalur menjadi lebih efisien. Library Flask digunakan sebagai framework backend yang menangani komunikasi antara frontend dan algoritma Dijkstra. Melalui Flask, sistem dapat menerima input pengguna, memproses pencarian rute, dan mengembalikan hasil dalam format JSON.

Analisis :

Pemilihan heapq menunjukkan bahwa implementasi Dijkstra dilakukan secara manual tanpa bantuan library graf seperti NetworkX. Pendekatan ini memberikan kontrol penuh terhadap proses pencarian jalur dan memperlihatkan implementasi algoritma secara langsung pada tingkat kode program.

2.2 Implementasi Algoritma Dijkstra

Source Code

```
distances = {node: float('inf') for node in graph}
```

```
distances[start] = 0
```

```
queue = [(0, start)]
```

```
previous_nodes = {  
    node: None  
    for node in graph  
}
```

Penjelasan :

Pada tahap awal, seluruh node diberikan nilai jarak tak hingga (infinity) untuk menandakan bahwa belum ada jalur yang diketahui menuju node tersebut. Node awal diberikan nilai 0 karena jarak dari titik awal ke dirinya sendiri adalah nol. Variabel queue digunakan sebagai priority queue yang akan selalu memproses node dengan jarak terkecil terlebih dahulu.

Analisis :

Tahap inisialisasi merupakan fondasi utama algoritma Dijkstra. Dengan memberikan nilai tak hingga pada seluruh node, sistem dapat membandingkan dan memperbarui jarak secara bertahap hingga ditemukan jalur optimal. Struktur ini memungkinkan algoritma bekerja secara sistematis tanpa perlu mengevaluasi seluruh kemungkinan jalur seperti metode brute force.

2.3 Pembaruan Jarak Tetangga

Source Code

```
for neighbor, weight in graph[current_node].items():  
    distance = current_distance + weight  
    if distance < distances[neighbor]:  
        distances[neighbor] = distance  
        previous_nodes[neighbor] = current_node  
        heapq.heappush(  
            queue,  
            (distance, neighbor)  
        )
```

Penjelasan :

Bagian ini melakukan proses relaksasi (relaxation), yaitu memperbarui jarak suatu node apabila ditemukan jalur yang lebih pendek melalui node yang sedang diproses. Jika jarak

baru lebih kecil daripada jarak sebelumnya, maka nilai jarak diperbarui dan node tersebut dimasukkan kembali ke priority queue.

Analisis :

Proses relaksasi merupakan inti dari algoritma Dijkstra. Keakuratan jalur yang dihasilkan sangat bergantung pada mekanisme ini karena setiap alternatif jalur akan dibandingkan secara terus-menerus hingga diperoleh bobot minimum. Dengan pendekatan tersebut, sistem dapat menjamin bahwa rute yang dipilih merupakan jalur terpendek berdasarkan bobot graf yang telah ditentukan.

2.4 Representasi Graf Kampus

Source Code

```
GRAF_UNIB = {  
    "Rektorat": {  
        "Simp_Rektorat_Utama": 35  
    },  
    "Simp_Rektorat_Utama": {  
        "Rektorat": 35,  
        "Simp_Layanan_Terpadu": 100,  
        "Tikungan_Rektorat_Fisip_1": 110  
    }  
}
```

Penjelasan :

Graf disimpan dalam bentuk dictionary bertingkat (adjacency list). Setiap node memiliki daftar node tetangga beserta bobot jaraknya.

Sebagai contoh, node Rektorat terhubung dengan Simp_Rektorat_Utama sejauh 35 meter.

Analisis :

Penggunaan adjacency list sangat sesuai untuk graf kampus karena jumlah hubungan antar node relatif lebih sedikit dibandingkan seluruh kemungkinan koneksi yang dapat terbentuk. Struktur ini membuat penggunaan memori lebih efisien dibandingkan adjacency matrix dan mempercepat proses pencarian tetangga selama eksekusi algoritma.

2.5 Endpoint API Pencarian Rute

Source Code

```
@app.route('/api/rute-terpendek')  
def api_rute_terpendek():
```

Penjelasan :

Endpoint ini berfungsi sebagai layanan utama yang menerima lokasi awal dan tujuan dari frontend. Data yang diterima kemudian diproses menggunakan algoritma Dijkstra untuk menghasilkan rute terpendek.

Analisis :

Endpoint ini menjadi penghubung utama antara antarmuka pengguna dan proses komputasi backend. Dengan menggunakan pendekatan API, sistem menjadi lebih modular karena logika algoritma dipisahkan dari tampilan antarmuka.

2.6 Visualisasi Jalur pada Leaflet

Source Code

```
routingKontrolAktif =  
  L.Routing.control({  
    waypoints: [  
      L.latLng(...),  
      L.latLng(...)  
    ]  
  }).addTo(map);
```

Penjelasan :

Kode ini digunakan untuk menampilkan jalur hasil perhitungan pada peta Leaflet. Jalur divisualisasikan dalam bentuk garis yang menghubungkan titik awal dan tujuan sehingga pengguna dapat melihat rute secara langsung.

Analisis :

Visualisasi merupakan komponen penting dalam sistem navigasi. Tanpa visualisasi, hasil algoritma hanya berupa daftar node yang sulit dipahami pengguna. Integrasi Leaflet membuat hasil pencarian lebih informatif dan meningkatkan pengalaman pengguna dalam menggunakan sistem.

1. 2. Analisis Permasalahan

Permasalahan utama dalam sistem ini adalah bagaimana menentukan rute terpendek antar lokasi di lingkungan Universitas Bengkulu secara efisien dan akurat. Lingkungan kampus memiliki banyak titik lokasi yang saling terhubung melalui jaringan jalan yang kompleks, sehingga proses pencarian jalur secara manual menjadi tidak efektif.

Selain itu, pengguna membutuhkan sistem yang dapat memberikan informasi rute secara cepat tanpa harus memahami struktur jalan secara keseluruhan. Oleh karena itu, diperlukan sebuah sistem yang mampu memodelkan area kampus ke dalam bentuk graf serta melakukan perhitungan otomatis untuk menemukan jalur terbaik. Permasalahan lain yang muncul adalah bagaimana sistem dapat menampilkan hasil perhitungan dalam bentuk visual yang

mudah dipahami pengguna, sehingga tidak hanya bersifat komputasional tetapi juga informatif secara visual.

3. Analisis Algoritma Dijkstra

Algoritma Dijkstra digunakan sebagai metode utama dalam menyelesaikan permasalahan pencarian rute terpendek pada sistem ini. Algoritma ini bekerja pada graf berbobot dengan cara mencari jalur dengan total bobot terkecil dari titik awal ke titik tujuan. Dalam implementasinya, setiap lokasi pada kampus direpresentasikan sebagai node, sedangkan jalan penghubung antar lokasi direpresentasikan sebagai edge dengan bobot berupa jarak.

3.1 Prinsip Kerja Algoritma

Algoritma Dijkstra bekerja dengan prinsip greedy, yaitu selalu memilih node dengan jarak sementara terkecil untuk diproses terlebih dahulu. Proses ini dilakukan secara berulang hingga semua node diproses atau tujuan telah ditemukan.

Tahapan Analisis :

1. Inisialisasi

Semua node diberi nilai jarak awal tak hingga, kecuali node awal yang bernilai nol.

2. Pemilihan Node Terdekat

Node dengan jarak terkecil dipilih sebagai node aktif untuk dievaluasi.

3. Relaksasi (Relaxation)

Sistem mengecek apakah terdapat jalur yang lebih pendek menuju node tetangga, kemudian memperbarui jarak jika ditemukan nilai yang lebih kecil.

4. Pencatatan Jalur

Setiap pembaruan jarak disertai dengan pencatatan node sebelumnya untuk membentuk rute.

5. Rekonstruksi Jalur

Setelah tujuan tercapai, jalur terpendek dibentuk kembali dari node tujuan ke node awal.

4. Analisis Properti Graf

4.1 Struktur Graf pada Sistem

Sistem navigasi yang dikembangkan menggunakan struktur graf berbobot untuk merepresentasikan jaringan jalan di lingkungan Universitas Bengkulu. Setiap lokasi seperti Rektorat, Perpustakaan, Fakultas Teknik, dan gedung lainnya direpresentasikan sebagai simpul (node), sedangkan hubungan antar lokasi direpresentasikan sebagai sisi (edge) yang

memiliki bobot berupa jarak tempuh dalam satuan meter. Selain node utama berupa gedung dan fasilitas kampus, sistem juga memanfaatkan node tambahan seperti simpang jalan dan tikungan. Penambahan node tersebut bertujuan agar jalur yang dihasilkan mengikuti kondisi jalan yang sebenarnya sehingga proses navigasi menjadi lebih realistis. Dengan struktur graf tersebut, algoritma Dijkstra dapat melakukan pencarian jalur secara sistematis berdasarkan hubungan antar node yang telah didefinisikan.

Source Code

```
GRAF_UNIB = {  
  
    "Rektorat": {  
        "Simp_Rektorat_Utama": 35,  
    },  
  
    "Simp_Rektorat_Utama": {  
        "Rektorat": 35,  
        "Simp_Layanan_Terpadu": 100,  
        "Tikungan_Rektorat_Fisip_1": 110  
    }  
}
```

Penjelasan :

Struktur graf disimpan menggunakan adjacency list berbentuk dictionary bersarang. Setiap node memiliki daftar node tetangga beserta bobot jalurnya.

Analisis :

Penggunaan adjacency list sangat sesuai untuk implementasi graf kampus karena jumlah koneksi antar node relatif sedikit dibandingkan seluruh kemungkinan hubungan yang dapat terbentuk. Pendekatan ini lebih efisien dalam penggunaan memori dibandingkan adjacency matrix dan mempercepat proses pencarian tetangga saat algoritma berjalan.

4.2 Node (Vertex)

Node merupakan representasi lokasi yang terdapat dalam lingkungan Universitas Bengkulu.

Jenis Node :

- a. Gedung Utama
 - 1) Rektorat
 - 2) Perpustakaan
 - 3) FMIPA
 - 4) Teknik

5) FKIP_UNIB

6) FISIP

7) Hukum

8) Kedokteran

b. Fasilitas Pendukung

1) Danau

2) GSG

3) PKM

4) UPA_TIK

5) Baitul_Hikmah

c. Node Penghubung

1) Simp_Rektorat_Utama

2) Simp_Perpustakaan

3) Simp_Teknik_Gsg

4) Simp_GB_Cluster

d. Node Tikungan

1) Tikungan_FKIP

2) Tikungan_Stadion

3) Tikungan_GB

Analisis :

Penambahan node simpang dan tikungan menjadi salah satu keunggulan sistem karena mampu menggambarkan kondisi jalan kampus secara lebih realistis. Jika hanya menggunakan node gedung, maka jalur yang dihasilkan cenderung berupa garis lurus yang tidak sesuai dengan kondisi sebenarnya.

4.3 Edge (Sisi)

Edge merupakan jalur penghubung antar node.

Source Code

```
"Simp_Teknik_Gsg": {  
    "Simp_Selatan_Fisip": 140,  
    "Teknik": 40,  
    "GSG": 50,  
    "Tikungan_Stadion": 90  
}
```


Penjelasan :

Node Simp_Teknik_Gsg memiliki empat jalur yang dapat dilalui menuju lokasi lain dengan bobot yang berbeda.

Analisis :

Semakin banyak edge yang dimiliki suatu node, semakin banyak alternatif rute yang dapat dipertimbangkan oleh algoritma Dijkstra. Hal ini meningkatkan fleksibilitas sistem dalam menentukan jalur terbaik menuju tujuan.

4.4 Bobot Graf

Bobot graf merepresentasikan jarak antar node dalam satuan meter.

Contoh:

No	Jalur	Jarak
1	Rektorat → Simp_Rektorat_Utama	35 m
2	Simp_Perpustakaan → FKIP_Dekanat	75 m
3	Teknik → Lab_Terpadu_Teknik	30 m

Analisis :

Bobot menjadi faktor utama dalam proses pengambilan keputusan oleh algoritma Dijkstra. Jalur dengan total bobot terkecil akan dipilih sebagai rute optimal. Oleh karena itu, akurasi bobot sangat memengaruhi kualitas hasil yang diberikan sistem.

5. Kompleksitas Algoritma**5.1 Analisis Kompleksitas Waktu**

Implementasi algoritma Dijkstra menggunakan priority queue melalui library heapq.

Kompleksitas waktunya adalah:

$$O((V + E)\log V)$$

Keterangan:

- V = jumlah node
- E = jumlah edge

Komponen memori yang digunakan:

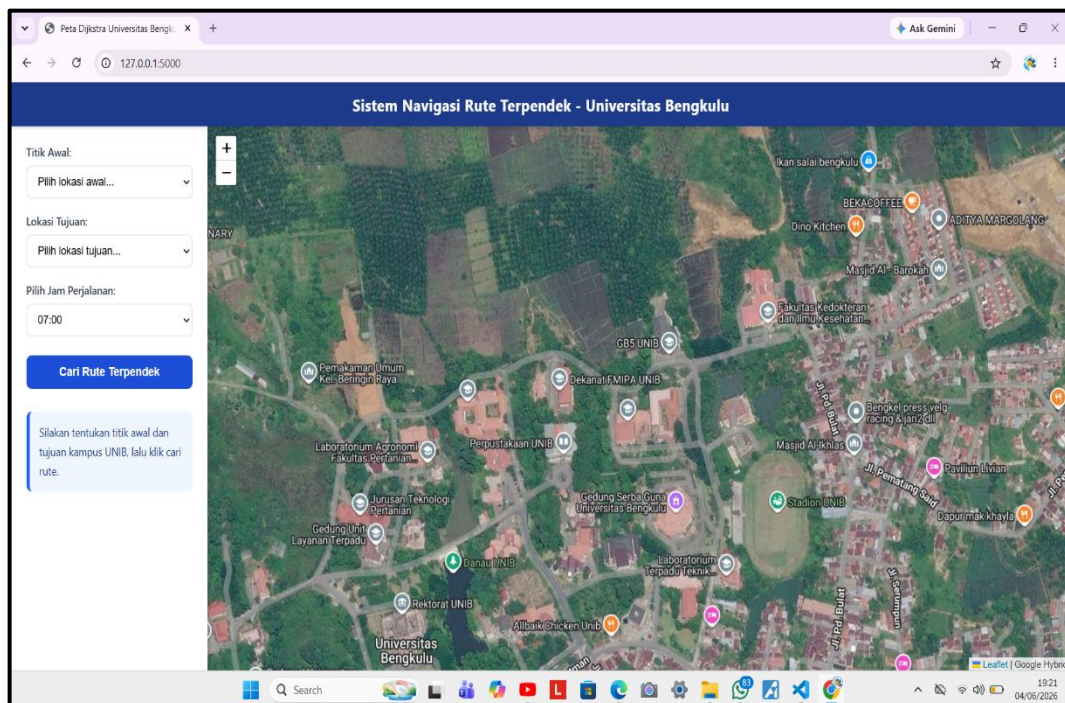
1. Penyimpanan graf.
2. Priority queue.
3. Dictionary jarak.
4. Dictionary previous node.
5. Jalur hasil pencarian.

Analisis :

Kebutuhan memori meningkat seiring bertambahnya node dan edge. Namun pada implementasi kampus, penggunaan memori masih sangat ringan dan tidak menimbulkan kendala performa. dengan V menyatakan jumlah node dan E menyatakan jumlah edge pada graf.

Kompleksitas tersebut menunjukkan bahwa waktu pencarian dipengaruhi oleh jumlah lokasi dan jumlah hubungan antar lokasi yang terdapat dalam graf. Karena cakupan sistem masih berada pada lingkungan Universitas Bengkulu dengan jumlah node yang relatif terbatas, proses pencarian jalur dapat dilakukan dengan sangat cepat dan responsif.

6. Analisis Penggunaan Sistem

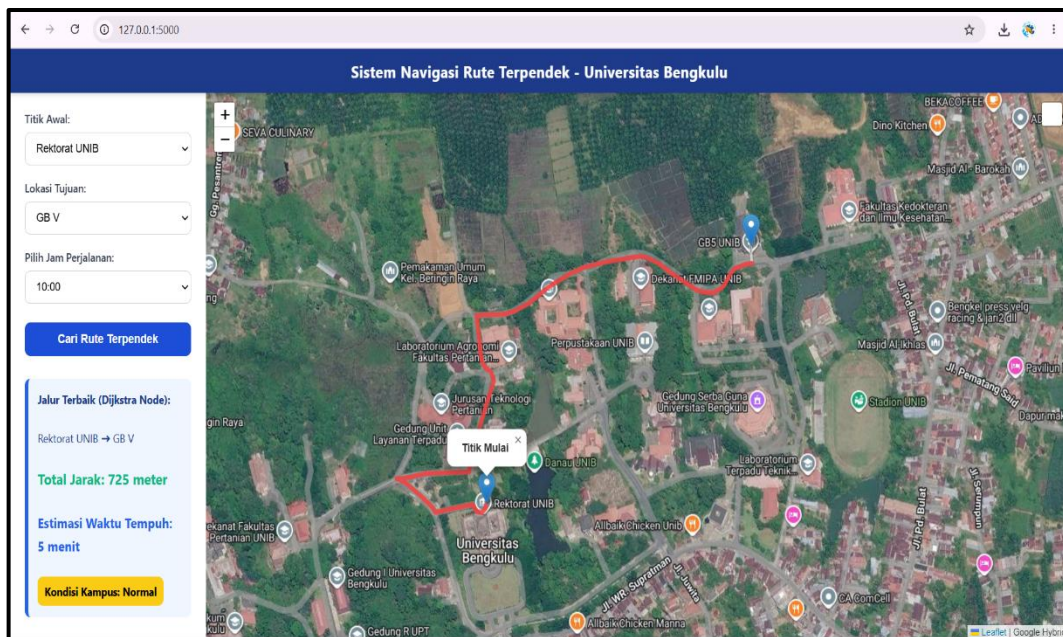


Gambar 1. Analisis Antarmuka Awal Sistem

Penjelasan :

Gambar diatas menunjukkan antarmuka pengguna (*user interface*) saat sistem navigasi pertama kali dijalankan (kondisi *default*). Pada halaman utama ini, pengguna disajikan dengan peta interaktif Universitas Bengkulu menggunakan *basemap* satelit yang ditenagai oleh Leaflet dan Google Hybrid. Di sisi kiri, terdapat panel kontrol navigasi yang responsif, yang menyediakan tiga fitur input utama: menu *dropdown* untuk memilih "Titik Awal", menu *dropdown* untuk memilih "Lokasi Tujuan", serta seleksi "Jam Perjalanan" untuk estimasi kondisi lalu lintas kampus. Pada kondisi awal ini, peta belum menampilkan jalur apa pun, dan panel informasi di bagian bawah memberikan instruksi kepada pengguna untuk menentukan titik awal dan tujuan sebelum memulai pencarian rute.

Dari perspektif arsitektur sistem, rancangan antarmuka awal ini mengintegrasikan komponen *front-end* dengan manajemen *state* yang terstruktur. Peta interaktif dimuat secara dinamis dengan memetakan koordinat geografis kampus Universitas Bengkulu ke dalam *viewport* pengguna. Sementara itu, elemen *input* pada panel kontrol telah diinisialisasi untuk terhubung dengan data lokasi pada basis data yang merepresentasikan *node-node* strategis dalam graf. Penyediaan opsi "Jam Perjalanan" juga mengindikasikan bahwa sistem siap mempertimbangkan variabel waktu dinamis sebelum algoritma mengeksekusi matriks ketetanggaan (*adjacency matrix*) yang tersedia.



Gambar 2. Tampilan Hasil Pencarian Rute dari Rektorat UNIB ke GB V

Penjelasan :

Gambar menampilkan hasil analisis spasial dan pengujian fungsionalitas algoritma Dijkstra pada skenario uji pertama, yaitu pencarian rute dari titik asal "Rektorat UNIB" menuju "GB V" yang disimulasikan pada pukul 10:00. Setelah menekan tombol eksekusi, sistem secara instan mengalkulasi bobot terkecil dari graf yang tersedia. Hasil pencarian divisualisasikan secara *real-time* berupa garis poli (*polyline*) tebal berwarna merah yang dengan presisi memetakan jalur sepanjang jalan akses internal kampus, ditandai dengan posisi *marker* "Titik Mulai" di area Rektorat.

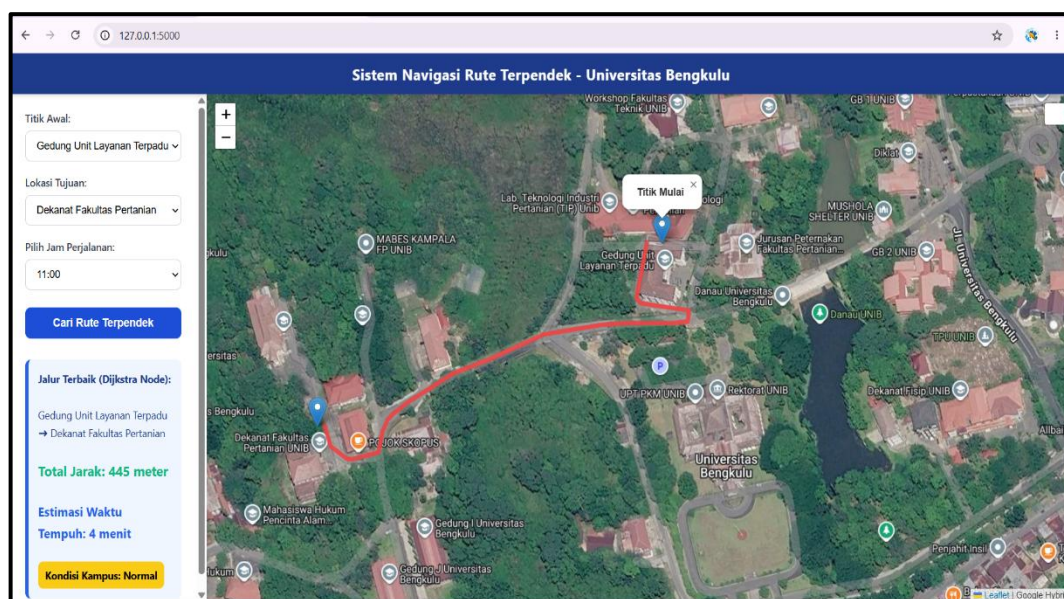
Hasil kalkulasi pada panel informasi menunjukkan efisiensi algoritma dalam memproses rute langsung (Rektorat UNIB → GB V) dengan akumulasi Total Jarak sebesar 725 meter dan prediksi Estimasi Waktu Tempuh 5 menit. Status indikator "Kondisi Kampus: Normal" menunjukkan bahwa pada jam tersebut, parameter pembobot waktu tidak mengalami penalaran penalti (kemacetan), sehingga algoritma dapat merekomendasikan rute utama ini sebagai jalur paling optimal secara matematis.



Gambar 3. Tampilan Hasil Pencarian Rute dari GB I ke Gedung Serba Guna (GSG)

Penjelasan :

Gambar memuat visualisasi hasil pengujian pada skenario uji kedua dengan kompleksitas *multi-node*, yang menghubungkan titik awal "GB I" menuju "Gedung Serba Guna (GSG)" pada waktu simulasi pukul 11:00. Berbeda dengan uji coba sebelumnya, pada skenario ini algoritma Dijkstra berhasil menganalisis dan memecah rute menjadi beberapa segmen graf bertahap. Hal ini dibuktikan pada panel informasi yang menampilkan urutan perjalanan secara berantai (*Dijkstra Node*), yaitu melintasi GB I → Dekanat FKIP → Gedung Serba Guna (GSG). Sistem secara akurat menjumlahkan bobot edge dari jaringan jalan lokal yang dilewati (termasuk visualisasi jalur melintasi Jl. Universitas Bengkulu) dan menghasilkan Total Jarak yang lebih pendek, yaitu 415 meter dengan Estimasi Waktu Tempuh hanya 4 menit.



Gambar 4. Tampilan Hasil Pencarian Rute dari Gedung Unit Layanan Terpadu ke Dekanat Fakultas Pertanian

Penjelasan :

Pada Gambar diatas ditampilkan hasil pencarian rute terpendek dari Gedung Unit Layanan Terpadu menuju Dekanat Fakultas Pertanian. Sistem berhasil menghitung jalur menggunakan algoritma Dijkstra dan menampilkan hasilnya pada peta digital. Jalur ditunjukkan dengan garis berwarna merah yang menghubungkan lokasi awal dan tujuan. Selain visualisasi rute, sistem juga menampilkan informasi jarak tempuh sebesar 445 meter, estimasi waktu perjalanan 4 menit, serta kondisi kampus pada jam yang dipilih yaitu Normal. Hasil ini menunjukkan bahwa sistem mampu memberikan informasi navigasi secara jelas dan mudah dipahami oleh pengguna.



Gambar 5. Tampilan Hasil Pencarian Rute dari Fakultas Kedokteran ke Masjid Baitul Hikmah UNIB

Penjelasan :

Pada Gambar diatas ditampilkan hasil pencarian rute terpendek dari Fakultas Kedokteran menuju Masjid Baitul Hikmah UNIB. Setelah pengguna menentukan titik awal dan tujuan, sistem memproses data menggunakan algoritma Dijkstra untuk memperoleh jalur dengan jarak paling minimum. Hasil perhitungan divisualisasikan pada peta dalam bentuk garis berwarna merah yang menunjukkan lintasan perjalanan. Sistem juga menampilkan informasi jarak tempuh sebesar 495 meter, estimasi waktu perjalanan 4 menit, serta kondisi kampus Normal. Tampilan ini membuktikan bahwa sistem dapat membantu pengguna menemukan rute menuju lokasi tujuan secara cepat dan interaktif.

7. Analisis Pengujian Sistem

7.1 Pengujian Jalur Dekat

Proses pencarian jalur dimulai ketika pengguna memilih lokasi awal dan lokasi tujuan pada antarmuka sistem. Data tersebut kemudian dikirim ke server untuk diproses

menggunakan algoritma Dijkstra. Pada tahap awal, sistem memberikan nilai jarak tak hingga kepada seluruh node yang terdapat pada graf. Selanjutnya node awal diberikan nilai jarak nol dan dimasukkan ke dalam priority queue sebagai titik awal proses pencarian. Sistem kemudian akan memeriksa node dengan jarak terkecil, mengevaluasi seluruh node tetangganya, dan memperbarui jarak apabila ditemukan jalur yang lebih pendek.

Proses tersebut berlangsung secara berulang hingga node tujuan berhasil ditemukan. Setelah tujuan ditemukan, sistem melakukan penelusuran kembali terhadap node-node yang telah dilalui sehingga terbentuk jalur terpendek dari lokasi awal menuju lokasi tujuan. Pendekatan ini memungkinkan sistem menghasilkan rute yang memiliki total jarak minimum dibandingkan seluruh alternatif jalur yang tersedia pada graf.

7.2 Pengujian Validasi Input

Source Code

```
if dari not in GRAF_UNIB or ke not in GRAF_UNIB:  
    return jsonify(  
        "status": "error",  
        "message": "Lokasi tidak valid"  
    ), 400
```

Penjelasan :

Backend melakukan validasi terhadap lokasi yang dipilih pengguna.

Analisis :

Validasi diperlukan untuk mencegah sistem memproses node yang tidak terdapat dalam graf. Dengan demikian stabilitas sistem dapat terjaga.

8. Evaluasi Sistem

Berdasarkan hasil implementasi dan pengujian yang dilakukan, sistem berhasil menjalankan fungsi utama berupa pencarian rute terpendek menggunakan algoritma Dijkstra. Jalur yang dihasilkan sesuai dengan struktur graf yang telah dibangun dan mampu memberikan rekomendasi perjalanan yang optimal berdasarkan total jarak minimum. Selain itu, integrasi dengan Leaflet.js memungkinkan hasil pencarian divisualisasikan secara interaktif sehingga memudahkan pengguna dalam memahami arah perjalanan. Fitur estimasi waktu tempuh juga memberikan nilai tambah karena mampu memberikan informasi perjalanan yang lebih informatif. Secara keseluruhan, sistem telah memenuhi tujuan pengembangan sebagai media navigasi digital di lingkungan Universitas Bengkulu yang mampu membantu pengguna menemukan jalur terpendek menuju lokasi tujuan secara efektif dan efisien.

8.1 Kelebihan Sistem

1) Implementasi Algoritma Dijkstra Secara Langsung

Sistem mengimplementasikan algoritma Dijkstra secara manual tanpa menggunakan library graf tambahan. Hal ini membuat proses perhitungan lebih mudah dipahami dan dianalisis.

2) Representasi Jalan Lebih Realistis

Penggunaan node simpang, tikungan, dan bundaran membuat jalur yang dihasilkan lebih sesuai dengan kondisi jalan kampus sebenarnya.

3) Integrasi Web yang Baik

Integrasi Flask dan Leaflet memungkinkan hasil pencarian ditampilkan secara interaktif dan mudah dipahami.

4) Estimasi Waktu Tempuh

Selain menghitung jarak terpendek, sistem juga memberikan estimasi waktu perjalanan berdasarkan jam aktivitas kampus yang dipilih pengguna. Pada jam masuk kuliah, jam istirahat, dan jam pulang, sistem memberikan tambahan waktu tertentu untuk mensimulasikan kondisi kampus yang lebih ramai. Sebaliknya, pada jam-jam yang relatif sepi, estimasi waktu yang diberikan menjadi lebih singkat. Meskipun pendekatan ini masih menggunakan nilai estimasi statis dan belum memanfaatkan data kepadatan secara real-time, fitur tersebut mampu memberikan gambaran perjalanan yang lebih realistis dibandingkan hanya menampilkan jarak tempuh.

5) Visualisasi Spasial Berbasis Citra Satelit Relevan

Penggunaan *basemap* satelit Google Hybrid yang diintegrasikan ke dalam peta interaktif Leaflet memberikan representasi visual yang sangat akurat. Pengguna dapat dengan mudah mengidentifikasi gedung, vegetasi, dan karakteristik fisik area Universitas Bengkulu secara langsung, sehingga mempermudah orientasi spasial dibandingkan hanya menggunakan peta berbasis vektor biasa.

6) Efisiensi Manajemen State dan Pengolahan Graf Dinamis

Sistem dirancang dengan pemisahan logika yang jelas antara panel kontrol *front-end* dan pemrosesan matriks ketetanggaan (*adjacency matrix*) di sisi *back-end*. Ketika pengguna mengubah parameter input (seperti titik awal atau tujuan), sistem dapat merespons secara instan tanpa perlu memuat ulang seluruh halaman (*no full-page reload*), sehingga komputasi rute terasa sangat cepat dan responsif.

7) Kemudahan Skalabilitas Data Node Navigasi

Arsitektur jaringan graf yang dibangun memungkinkan penambahan, pengurangan, atau modifikasi koordinat lokasi (*node*) serta jalur penghubung (*edge*) di masa

mendatang secara fleksibel. Karakteristik ini membuat sistem sangat siap untuk dikembangkan lebih lanjut, baik untuk perluasan cakupan wilayah navigasi kampus maupun untuk penyesuaian tata letak infrastruktur gedung baru.

8.2 Keterbatasan Sistem

1) Bobot Masih Statis

Bobot jarak ditentukan secara manual sehingga belum dapat menyesuaikan perubahan kondisi jalan secara otomatis.

2) Belum Mendukung GPS

Sistem belum dapat mendeteksi posisi pengguna secara real-time.

3) Belum Mempertimbangkan Kemacetan

Algoritma hanya mempertimbangkan jarak, belum mempertimbangkan tingkat kepadatan lalu lintas.

4) Graf Masih Terbatas

Cakupan sistem masih berada pada lingkungan Universitas Bengkulu sehingga belum dapat digunakan di luar area kampus.

5) Akurasi Estimasi Waktu Berbasis Aturan Statis

Meskipun sistem sudah memiliki fitur estimasi waktu dinamis berdasarkan jam aktivitas kampus, penambahan bobot waktu tersebut masih menggunakan logika aturan statis (*hard-coded rules*). Sistem belum mampu melakukan penyesuaian dinamis terhadap kejadian tak terduga yang terjadi secara mendadak di lapangan, seperti penutupan jalan sementara karena acara kampus atau cuaca buruk.

9. Kesimpulan Analisis

Berdasarkan hasil analisis terhadap source code, struktur graf, kompleksitas algoritma, serta pengujian sistem, dapat disimpulkan bahwa implementasi algoritma Dijkstra pada sistem navigasi Universitas Bengkulu telah berjalan dengan baik. Representasi graf menggunakan adjacency list mampu menggambarkan jaringan jalan kampus secara efektif, sedangkan penggunaan priority queue melalui heapq membuat proses pencarian jalur berlangsung efisien.

Hasil pengujian menunjukkan bahwa sistem mampu menghasilkan rute terpendek secara konsisten pada berbagai skenario pencarian. Integrasi antara Flask sebagai backend dan Leaflet.js sebagai frontend juga memungkinkan visualisasi rute dilakukan secara interaktif. Dengan kompleksitas waktu $O((V+E) \log V)$, sistem memiliki performa yang sangat baik untuk ukuran graf kampus dan layak digunakan sebagai solusi navigasi digital di lingkungan Universitas Bengkulu.